

Practical - 5

Practical Title: Registration Form with Frontend Validation and User-Friendly Error Handling

Problem Definition: Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations.

.

A. Theory

1. Are correct input types and attributes used?

Answer: Yes. Appropriate HTML5 input types should be used for each field to improve validation and user experience.

HTML provides input types such as text, email, password, tel, and number. Attributes such as required, pattern, min, max, and maxlength help validate user input before form submission.

Reference Link:

https://www.w3schools.com/HTML/html_form_input_types.asp

https://www.w3schools.com/html/html_form_attributes.asp

2 Are correct input types and attributes used?

Answer: Yes. Error messages should be displayed directly below or near the संबंधित input field so that the user can easily identify and correct the mistake.

For example:

- Name field → "Please enter a valid name."
- Email field → "Please enter a valid email address."
- Mobile field → "Mobile number must contain 10 digits."
- Password field → "Password does not meet the required criteria."
- Confirm Password → "Passwords do not match."
- Terms checkbox → "Please accept the terms and conditions."

JavaScript can dynamically validate fields and display specific error messages. The form submission should be prevented until all fields contain valid data.

Reference Link:

https://www.w3schools.com/js/js_project_form_validation.asp

3. Is password strength checked?

Answer: Yes. Password strength should be checked using JavaScript and/or Regular Expressions.

A strong password can require:

- At least 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one number
- Optionally, one special character

Example Regular Expression:

```
/^(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}$/
```

The password can also be classified as:

- Weak
- Medium
- Strong

A password strength meter provides immediate feedback to the user while entering the password. W3Schools demonstrates password validation using regular expressions and checks for lowercase letters, uppercase letters, numbers, and minimum length

Reference Link:

https://www.w3schools.com/howto/howto_js_password_validation.asp

https://www.w3schools.com/TAg/att_pattern.asp

4. Is the form accessible using keyboard and screen-reader-friendly labels?

Answer:

Yes. The registration form should be designed to be accessible for keyboard users and screen-reader users.

Accessibility can be improved by:

1. Using <label> elements with the for attribute.
2. Connecting each label with its corresponding input using the id attribute.
3. Using proper keyboard navigation with the Tab key.
4. Providing visible focus indicators.
5. Using <fieldset> and <legend> for related fields such as gender.
6. Using aria-describedby where additional instructions or error messages are required.

7. Avoiding validation messages that depend only on color.

Example:

```
<label for="email">Email Address:</label>
```

```
<input type="email" id="email" name="email" required>
```

Labels are especially important because screen readers announce the label associated with an input field. Proper focus styling also helps keyboard users navigate the form.

Reference Link:

https://www.w3schools.com/accessibility/accessibility_labels.php

B. Implementation/ Output/Result

CSS:

```
C. /* =====
D.   Student Registration Form
E.   ===== */
F. .signup-fieldset{
G.     width:450px;
H.     text-align:left;
I. }
J.
K. .signup-fieldset h3{
L.     text-align:center;
M.     margin-left:0;
N. }
O.
P. .error{
Q.     display:block;
R.     color:#d32f2f;
S.     font-size:13px;
T.     font-weight:bold;
U.     margin-top:5px;
V.     margin-bottom:10px;
W.     min-height:16px;
X. }
Y.
Z. .gender-group{
AA.    display:flex;
BB.    gap:20px;
CC.    margin-top:5px;
DD.    margin-bottom:5px;
```

```

EE.}
FF.
GG. .inline-label{
HH.    display:inline-flex !important;
II.    align-items:center;
JJ.    gap:6px;
KK.    font-weight:normal !important;
LL.    margin-top:0 !important;
MM.    font-size:16px;
NN.}
OO.
PP. .inline-label input{
QQ.    width:auto !important;
RR.    margin:0 !important;
SS.}
TT.
UU. .terms-group{
VV.    margin-top:10px;
WW.    margin-bottom:5px;
XX.}

```

Javascript:

```
let signupform=document.getElementById("signupform");
```

```
if(signupform)
```

```
{
```

```
    // Field references
```

```
    let fullname=document.getElementById("fullname");
```

```
    let email=document.getElementById("email");
```

```
    let mobile=document.getElementById("mobile");
```

```
    let course=document.getElementById("course");
```

```
    let year=document.getElementById("year");
```

```
    let crpassword=document.getElementById("createpassword");
```

```

let copassword=document.getElementById("confirmpassword");

let terms=document.getElementById("terms");

let signupMessage=document.getElementById("message");


// Regular expressions

let namePattern=/^[A-Za-z\s]{3,}$/;

let emailPattern=/^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;

let mobilePattern=/^[6-9][0-9]{9}$/;

let passwordPattern=/^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[@$!%*?&]).{8,}$/;


// Helper to show/clear a single field's error message

function setError(fieldId, msg)
{
    let errorEl=document.getElementById(fieldId+"Error");

    if(errorEl) errorEl.innerHTML=msg;
}


function clearAllErrors()
{
    document.querySelectorAll("#signupform .error").forEach(function(el)
    {
        el.innerHTML="";
    });
}

```

```
signupform.addEventListener("submit", function(event)
{
    event.preventDefault();
    clearAllErrors();

    let valid=true;
    let selectedGender=document.querySelector('input[name="gender"]:checked');

    // Name validation
    if(!namePattern.test(fullname.value.trim()))
    {
        setError("fullname", "Name must contain only alphabets/spaces and be at least 3
characters.");
        valid=false;
    }

    // Email validation
    if(!emailPattern.test(email.value.trim()))
    {
        setError("email", "Please enter a valid email address.");
        valid=false;
    }

    // Mobile validation
```

```
if(!mobilePattern.test(mobile.value.trim()))  
{  
    setError("mobile","Mobile number must be exactly 10 digits and start with 6, 7, 8  
or 9.");  
    valid=false;  
}
```

// Course validation

```
if(course.value=="")  
{  
    setError("course","Please select a course.");  
    valid=false;  
}
```

// Year validation

```
if(year.value=="")  
{  
    setError("year","Please select your year.");  
    valid=false;  
}
```

// Gender validation

```
if(!selectedGender)  
{  
    setError("gender","Please select your gender.");
```

```
        valid=false;
    }

    // Password validation
    if(!passwordPattern.test(crpassword.value))
    {
        setError("createpassword", "Password must be 8+ characters with uppercase,
lowercase, number and special character.");
        valid=false;
    }

    // Confirm password validation
    if(copassword.value=="")
    {
        setError("confirmpassword", "Please confirm the password.");
        valid=false;
    }
    else if(crpassword.value!=copassword.value)
    {
        setError("confirmpassword", "Passwords do not match.");
        valid=false;
    }

    // Terms validation
    if(!terms.checked)
```

```
{  
    setError("terms","You must accept the Terms and Conditions.");  
    valid=false;  
}  
  
if(!valid)  
{  
    signupMessage.innerHTML="Please fix the highlighted errors.";  
    signupMessage.style.color="#d32f2f";  
    return;  
}  
  
signupMessage.style.color="green";  
signupMessage.innerHTML="Account Created Successfully!";  
});  
}
```

C. Evidence of Work (Screenshots)

StudentHub

Home

Welcome to StudentHub

Student Registration

Full Name:

Devanshi Shah

Email:

Enter your email

Please enter a valid email address.

Mobile Number:

Enter 10-digit mobile number

Mobile number must be exactly 10 digits and start with 6, 7, 8 or 9.

Description:

This figure shows validation for mandatory fields. Although the name and email are entered, the mobile number, course, year, and gender are not properly selected or entered. Red error messages inform the user about the missing or invalid information

Devanshi Shah

Email:

25dce107@charusat.edu.in

Mobile Number:

Enter 10-digit mobile number

Mobile number must be exactly 10 digits and start with 6, 7, 8 or 9.

Course:

Select Course

Please select a course.

Year:

Select Year

Please select your year.

Gender:

Male Female Other

Please select your gender.

Create Password:

Description:

This figure shows validation for mandatory fields. Although the name and email are entered, the mobile number, course, year, and gender are not properly selected or entered. Red error messages inform the user about the missing or invalid information.

The image shows a registration form with the following elements and validation errors:

- Name:** A text input field with a red border and a red error message: "Name is required".
- Email:** A text input field with a red border and a red error message: "Email is required".
- Mobile Number:** A text input field with a red border and a red error message: "Mobile number is required".
- Course:** A dropdown menu with "Web Development Framework" selected.
- Year:** A dropdown menu with "2nd Year" selected.
- Gender:** Radio buttons for "Male", "Female" (selected), and "Other".
- Create Password:** A text input field with a red border and a red error message: "Password must be 8+ characters with uppercase, lowercase, number and special character."
- Confirm Password:** A text input field with a red border and a red error message: "Please confirm the password."
- Terms & Conditions:** A checkbox labeled "I accept the Terms and Conditions" which is unchecked. Below it is a red error message: "You must accept the Terms and Conditions."
- Register Button:** A blue button labeled "Register".

Description:

The user has selected the course, year, and gender, but the password fields and Terms & Conditions are not properly validated. The form displays messages such as:

- Password must contain at least 8 characters with uppercase, lowercase, number, and special character.
- Please confirm the password.
- You must accept the Terms and Conditions

2nd Year

Gender:
☐ Male ☒ Female ☐ Other

Create Password:

Confirm Password:

Passwords do not match.

☐ I accept the Terms and Conditions
You must accept the Terms and Conditions.

Register

Please fix the highlighted errors.

Description:

This figure shows the situation where the Password and Confirm Password fields contain different values. The system displays the error message **"Passwords do not match."** The Terms & Conditions checkbox is also unchecked, resulting in another validation message. A general message, **"Please fix the highlighted errors,"** is displayed after clicking the Register button.

Year:
2nd Year

Gender:
☐ Male ☒ Female ☐ Other

Create Password:

Confirm Password:

☒ I accept the Terms and Conditions

Register

Account Created Successfully!

Description:

This figure shows successful form submission after all fields are correctly filled, both passwords match, and the Terms & Conditions checkbox is selected. A green success message, "**Account Created Successfully!**", is displayed.

D. GitHub Repository Link:

<https://github.com/devanshirshah24-pixel/WDF-Practicals/tree/main/Practical5>

E. Challenges Faced:

1. Regular Expression Validation

Creating correct regular expressions for name, mobile number, and password validation was challenging. The patterns had to allow valid input while rejecting invalid characters.

2. Password and Confirm Password Matching

Comparing both password fields using JavaScript and displaying an appropriate error message required careful validation logic.

3. Displaying User-Friendly Error Messages

It was challenging to show errors near the respective input fields without making the form confusing or cluttered.

4. Preventing Invalid Form Submission

JavaScript validation had to ensure that the form was submitted only when all required fields contained valid data.

5. Password Strength Implementation

Checking uppercase letters, lowercase letters, numbers, and minimum password length required multiple validation conditions.

6. Accessibility

Making the form usable through keyboard navigation and understandable by screen readers required proper use of labels, focus indicators, and semantic HTML elements.

7. Testing Different Inputs

The form needed to be tested with both valid and invalid data to ensure that every validation rule worked correctly.

F. Learning Outcomes:

After completing this practical, the following concepts were learned:

1. **HTML5 Forms**
Learned how to create a student registration form using different HTML input types.
2. **Form Validation**
Learned to validate user input using HTML attributes such as required, pattern, min, and maxlength.
3. **JavaScript Validation**
Learned to perform client-side validation using JavaScript.
4. **Regular Expressions**
Learned to use Regular Expressions for validating names, mobile numbers, emails, and passwords.
5. **Password Strength Checking**
Learned how to check password complexity based on length, uppercase letters, lowercase letters, and numbers.
6. **Error Handling**
Learned to display clear and user-friendly error messages near the corresponding fields.
7. **Form Accessibility**
Learned the importance of labels, keyboard navigation, focus indicators, and screen-reader-friendly forms.
8. **Testing and Debugging**
Learned to test forms using valid and invalid inputs and debug validation errors using browser developer tools.

Faculty Remarks: (Keep this blank for faculty)